Building an Inference Engine - Notes

Companion to Part 1: Loading model weights. This is the loading milestone: later videos add the forward pass, generation, batching, and paging. The current loader still targets this fixed Llama 3.2 1B layout; it does not infer a generic architecture from the checkpoint.

SafeTensors

There are different formats you can download models in, SafeTensors being one of the most popular.

A safetensor file has 3 sections: **header size**, **JSON header**, **tensor data**. Quick aside: a tensor is a multi-dimensional array of numbers, and the tensor data here is the model weights.

- **Header size** is always 8 bytes, an unsigned 64-bit integer stored in little-endian order.
- **JSON header** is just key-value pairs, where the key is a unique tensor name. Every value is itself a JSON with 3 keys:
  - `dtype` - what data type the tensor is stored in
  - `shape` - the dimensions of the tensor
  - `data_offsets` - a start and exclusive end telling us where that tensor's raw numbers live in the data section

What we implement

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(128256, 2048)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=512, bias=False)
          (v_proj): Linear(in_features=2048, out_features=512, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=8192, bias=False)
          (up_proj): Linear(in_features=2048, out_features=8192, bias=False)
          (down_proj): Linear(in_features=8192, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
```

That's pretty much the entire architecture for Llama 3.2 1B Instruct. What it gives us is the operations we need to run plus the data shapes and types we need to use. What it *doesn't* give us is the order of operations or the dtype - it's just an object with code I don't really understand yet.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For the dtype we can look the model up on HuggingFace: the weights are **BF16**.

> **BF16** is a 16-bit floating point number with 1 sign bit, 8 exponent bits, and 7 fraction bits. FP32 also has 8 exponent bits, so both cover a similar magnitude range, up to about `3.4 × 10^38`; this is not the range `-256` to `255`. FP32 has 23 fraction bits, so BF16 represents fewer distinct values within that range and rounds more coarsely. Storing weights in BF16 halves their memory use compared with FP32. The engine often converts values to FP32 for reductions and other arithmetic, then rounds the stored output back to BF16.

Order of operations

We need the order of operations to produce correct output, since the model is just specific computations in a specific order. For that, search up the architecture - diagrams are good, Sebastian Raschka has a bunch. Let's figure out how I'm supposed to dissect the diagram.

Input text → tokens. A **tokenizer** converts text into pieces the model recognizes. It splits the input into smaller pieces (whole words, pieces of words, punctuation) and assigns each an ID. It can do this because it built a fixed vocab of token pieces during training, each mapping to an ID - the ID value itself is arbitrary. What if it wasn't trained on some input? Then it breaks it down as far as it needs to until the pieces are in its vocab, e.g. `["hyperglob"] -> ["hyper", "gl", "ob"]`.

Tokens → embeddings. Then we hit up a mega table of model weights, the **embedding table**, where the token ID is a row index. `(embed_tokens): Embedding(128256, 2048)` tells us there are 128,256 possible token IDs and 2048 learned weights per token, which form an **embedding vector**.

The pink box is a **transformer block** (also called a layer). There are 16 of them, with a shit ton of operations I don't know yet but will get a clear understanding of as I build more of the engine:

1. RMS Norm
2. Residual connection
3. Masked grouped-query attention:
   - Q projection
   - K projection
   - V projection
   - RoPE with Q projection
   - RoPE with K projection
   - Attention / attention scores
   - Causal mask
   - Softmax
   - Attention scores with V projection
   - O projection (output projection)
4. Residual connection add
5. RMS Norm
6. Feed forward (like in first neural networks, multilayer perceptron):
   - Gate projection (first linear layer)
   - Up projection (second linear layer)
   - SiLU activation - similar to ReLU but looks more like a sigmoid

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

- Down projection (third linear layer)

7. Residual connection add

Then a final RMS Norm, a Linear output, then Argmax. I truly don't got that much of a clue on these computations yet - ngl still just copying from the diagram and docs. Will learn as I go.

An aside on memory

Data lives on **host** or **device**.

- **Host** = the PC/CPU. Big slow memory (DRAM), separate from the CPU, plus the CPU's own tiny fast on-chip SRAM (L1/L2 cache) for quick access to recently used memory.
- **Device** = the GPU. Also has big slow memory (VRAM) and small fast SRAM.

We want the large computations on the GPU, which means it needs access to the weights, input tokens, and intermediate results - but the GPU can't directly touch host DRAM. So we copy: **CPU DRAM → GPU VRAM**.

At the low level that's not as easy as it sounds. On the CPU we create a variable and write to it, compute how many elements it holds, multiply by the size of the type to get total bytes. Then on the GPU we allocate memory so we have somewhere to put it, and copy it over. Now the data is in device VRAM and usable in GPU computations. CUDA has its own functions for this - `cudaMalloc` and `cudaMemcpy` - which are pretty similar to ones I worked with in C.

Loading the model

First step of inference is actually loading the model, i.e. copying weights from CPU to GPU memory like we talked about above. The model is a single `model.safetensors` file: header size, JSON header, tensor data (many tensors).

Opening the file. In C we'd use `fopen` or `open()`; in C++ we use `std::ifstream` - input file stream, an object that lets the program read data from the file. We pass `ios::binary` to say this is raw binary, so every byte is preserved exactly. Without it the compiler sometimes translates special chars or line endings, which corrupts bytes and makes the offsets wrong.

Why we read the header. The JSON header is metadata on every tensor in the model. There are 16 transformer layers, each with 9 tensors, and each tensor occupies some space in the file recorded by its offsets. We need this because at some point we have to use these weight values and we can't keep reading from the file - super slow. It has to go to VRAM. Reading (and printing) the header also shows us the naming convention for tensors, which lets us quickly find what we're looking for later.

Reading the header. The file starts with the header size as a `uint64_t` (8 bytes) telling us how big the JSON header is. We call `.read(destination, bytes_to_read)` on the opened file for those first 8 bytes. Now we know the JSON header size, so we call `.read` again for that many bytes - the file pointer is already sitting at the start of the JSON header (8 bytes in), so we don't have to handle that. `.read` needs a `char` destination buffer, i.e. a string, so we make a string of size `headersize` initialized to `\0` and read into it.

Parsing the header. We use the JSON library we imported to turn the header string into a JSON object with `json::parse`. We need the offsets for 2 things:

1. The **max offset** tells us how much space the tensor data takes up - we know the header size but not the size of the raw weights.
2. The **start offsets** are needed later to build pointers, since we'll allocate all this on the GPU and want pointers on the CPU so we can quickly access GPU memory.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

To store them we use a hashmap (`unordered_map` in C++) keyed by `string` with `uint64_t` values. We initialize `max_offset` to zero since we check it every iteration.

Then we use a structured-binding for loop over the name and `tensor_info` of each header item. This is where having printed the header pays off - you know exactly what you're looking for. The offsets live at `"data_offsets"`, which is an array of two values (start and end). Grab them with `data_offsets.at(...)`, convert to `uint64_t`, then `emplace` them into the hashmap and recompute `max_offset` by checking this tensor's end offset against the current max.

Reading the tensor data. Relatively simple thanks to `max_offset`: the file pointer is already at the start of the raw tensor data and we know how large it is, so just call `.read` again. Where do we store it? A `vector<char>` - that's what `.read` expects, and it matches the file's exact raw byte layout. The actual values are BF16, which is just two contiguous 8-bit chars.

Getting it onto the GPU. Allocate first with `cudaMalloc`, which reserves a block on the GPU and sets a pointer to its start - so before that we initialize a `void` pointer to null so it can be assigned. Then `cudaMemcpy` runs the actual copy. Pretty simple params since we've already made everything it needs.

Hashmap vs struct. Initially I made a hashmap of pointers, each pointing to where that weight lives in GPU memory. That's fine for the initial load, but during inference we go through all 16 layers, and this approach means string construction, hashing, and a hashmap lookup every time - slow. Instead, a **struct** holding all the potential values for a layer, indexed by layer number, gives a much quicker lookup and no string construction in the loop.

That just means handling the construction on initial load instead. So: we create BF16 pointers to specific values, and for the tensors that repeat in every transformer layer we make an array of 16 BF16 pointers, one per layer's version of that tensor. Then we loop through each layer, do the math to figure out where each pointer should point, and save it in the struct so it can be called from CPU memory later.

First we get the **base** - where the memory points to on the GPU, which `cudaMalloc` returned to us and which we store in the weights struct. Then we compute each tensor's location from the base plus the offset hashmap, for every tensor in every layer.

By the way: we need the weights outside this function, so we pass the struct by reference into the loading function.

The struct approach works because we're building this specifically for Llama 3.2 1B - we know the architecture, and we printed the JSON header to see how the tensors are named.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.